Lecture 5
Functions, Program files & Preprocessing Directives and Analog Signals

Week Organization

→Lectures
- → Thursdays, **12:30 – 14:15**, LK247
- → For all students

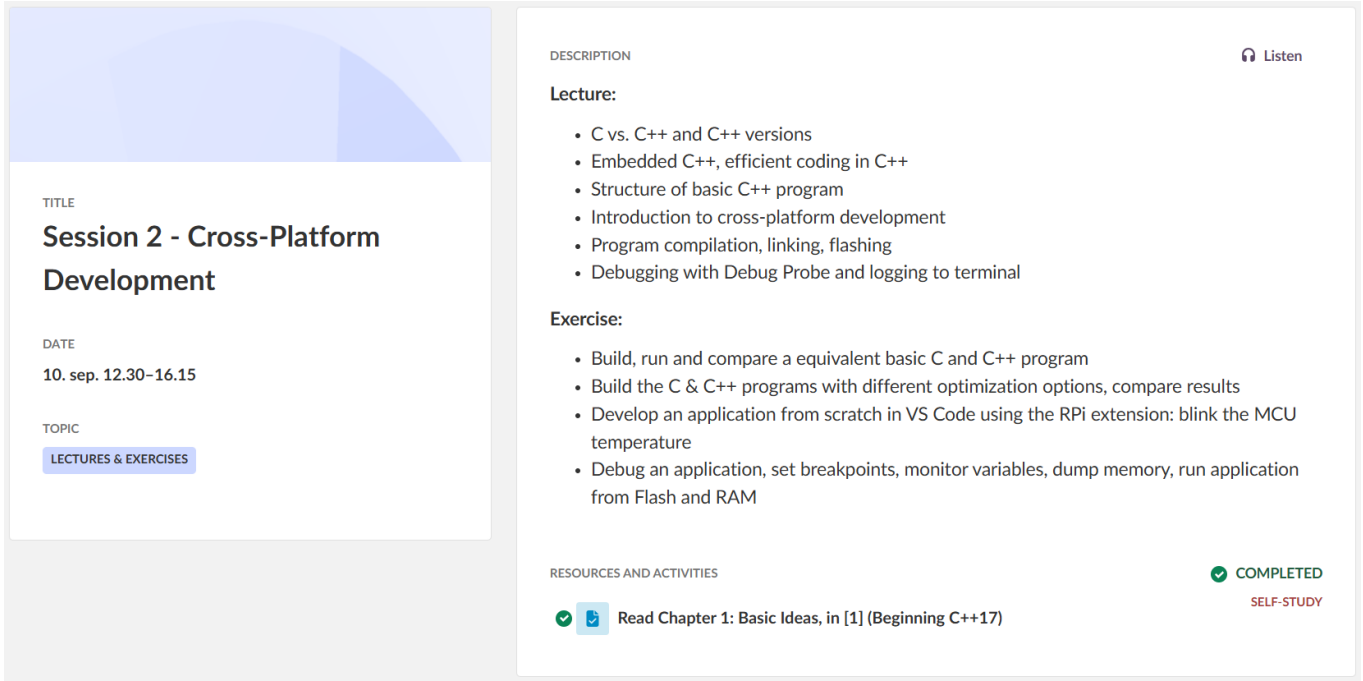
→Exercises/Labs
- → Thursdays, **14:30 – 16:15**, LK247
- → For all students
- → Group work (~2 students)
- → Using personal computers
- → On the RPi Pico kit from the library



Week Organization

→Lectures
- → Thursdays, **12:30 – 14:15**, LK247
- → For all students

→Exercises/Labs
- → Thursdays, **14:30 – 16:15**, LK247
- → For all students
- → Group work (~2 students)
- → Using personal computers
- → On the RPi Pico kit from the library



Detailed plan in **itslearning**, updated regularly

C++ Functions

Why functions matter?

→ A function is a named block of code that performs a well-defined task
→ Functions divide a large program into smaller, manageable units
→ Each function can be developed, tested, and reused independently
→ Execution starts in `main()`, which calls other functions
→ Active function calls are tracked using the call stack

→ Functions provide the basic structure for organizing C++ programs

```
int main()
{
    initialize();
    read_inputs();
    process_data();
    display_results();
}
```

Segmenting a Program

→ A real-world program typically consists of many functions

→ Benefits
- → Improves readability
- → Reduces code duplication
- → Simplifies testing and debugging
- → Supports reuse
- → Makes teamwork easier
- → Hides implementation details behind meaningful names

```
int main()
{
    initialize();
    read_inputs();
    process_data();
    display_results();
}
```

→ Good design principle: *A function should normally perform one clearly defined task*

Characteristics of a Function

→ A function is characterized by:
- → Its name
- → Its return type
- → The number and types of its parameters
- → Its function body

→ Here:
- → `double` is the return type
- → `power` is the function name
- → `x` and `n` are parameters
- → The statements between braces form the function body

```
double power(double x, int n)
{
    double result{1.0};

    for (int i{}; i < n; ++i)
        result *= x;

    return result;
}
```

Function Declaration and Definition

- Function declaration, or prototype
  - A declaration tells the compiler that a function exists
  - It specifies:
    - Function name
    - Return type
    - Parameter types

```
double power(double x, int n);
```

- Function definition
  - The definition provides the implementation
  - A function must be declared *before* it is called
  - Its definition can appear later in the source file

```
double power(double x, int n)
{
    // Function body
}
```

Function Body and Local Variables

→ Variables defined inside a function are local variables

→ Properties of local variables
- → Exist only while the function is executing
- → Are accessible only within their scope
- → Are normally stored in the current function's stack frame
- → Different function calls have separate local variables

```
double calculate_area(double w, double h)
{
    double area{w * h};
    return area;
}
```

→ When the function returns, its ordinary local variables cease to exist

Returning a Value

→ A function returns a result using `return`
→ The returned expression must be convertible to the declared return type

```
double square(double value)
{
    return value * value;
}
```

→ Functions returning nothing
  → Use `void` when no value is returned
  → A `return;` statement may be used to leave a `void` function early

```
void print_message()
{
    std::cout << "Ready\n";
}
```

Passing Arguments by Value

→ Pass-by-value gives the function a copy of the argument
  → `number` is still `10` after the call to `increment()`

→ Advantages
  → The original value is protected
  → The function has its own independent parameter
  → Appropriate for small, inexpensive types such as `int`, `char`, or `double`

→ Cost
  → Copying large objects may require additional time and memory.

```
void increment(int value)
{
    ++value;
}

int number{10};
increment(number);
```

Pointers as Parameters

→ A pointer parameter still receives its pointer by value, but the pointer can be used to access the original object
  → Now `number` becomes `11`

→ Important distinction
  → The pointer itself is copied
  → The copied pointer refers to the original object
  → A pointer may be `nullptr`, so validation may be required

→ Pointers are useful when *null* is meaningful or when working with low-level data and hardware interfaces

```
void increment(int* value)
{
    if (value)
        ++(*value);
}

int number{10};
increment(&number);
```

Pass-by-Reference

- → A reference parameter provides another name for the original argument
  - → Now `number` becomes 11
- → Advantages
  - → No explicit address-of or dereference syntax
  - → No copy of the complete object
  - → The function can modify the caller's object
- → Design warning
  - → A non-const reference indicates that the function may modify its argument

```
void increment(int& value)
{
    ++value;
}

int number{10};
increment(number);
```

References to const

→ Use a reference-to-const for efficient read-only access

→ Benefits
- → Avoids copying the object
- → Prevents the function from changing it
- → Can accept both const and non-const objects
- → Useful for larger objects such as strings and containers

→ General guideline
- → Use this form when the function only needs to inspect a large object

```
void print_message(const std::string& message)
{
    std::cout << message << '\n';
}
```

```
void process(const LargeObject& object);
```

Default Argument Values

→ A parameter can have a default value

```
double power(double x, int n = 2);
```

→ The function can then be called in two ways

```
power(5.0, 3);   // 5³
power(5.0);      // 5²
```

→ Rules
- → Default values are normally specified in the declaration
- → Parameters with defaults must appear at the end
- → If several parameters have defaults, arguments can only be omitted from right to left

```
void configure(int speed, int mode = 1, bool debug = false);
```

Returning Pointers and References

→ Functions may return pointers or references

```
int* find_value(int values[], std::size_t size, int target);
```

```
int& larger(int& a, int& b)
{
    return a > b ? a : b;
}
```

→ **Critical lifetime rule**
- → Never return a pointer or reference to an ordinary local variable
- → `value` no longer exists after the function returns, so the result is a *dangling reference*
- → A returned pointer or reference must refer to an object that remains alive

```
int& invalid()
{
    int value{42};
    return value;   // Dangerous
}
```

Return Values vs. Output Parameters

→ A function can communicate results in two ways

→ Return a *value*

→ Modify an *output parameter*

→ Preferred approach
- → Return the result when there is one natural output
- → Output parameters may be useful when:
  - → Several results must be produced
  - → Failure must be reported separately
  - → Existing interfaces require them

→ Modern C++ often replaces output parameters with returned objects, structures, or optional values

```
double calculate_average(
    const std::vector<double>& values);

bool calculate_average(
    const std::vector<double>& values,
    double& result);
```

Static Local Variables

→ A local variable declared static retains its value between calls
  → Each call returns the next number

```
unsigned next_id()
{
    static unsigned id{};
    return ++id;
}
```

→ Properties
  → Initialized only once
  → Exists for the remainder of the program
  → Accessible only inside the function
  → Preserves state across calls

→ Static locals avoid repeated initialization, but introduce hidden persistent state and require care with concurrency, interrupts, and reentrancy

Inline Functions

→ An inline function may be expanded at the place where it is called

→ Potential benefit
  → Avoids some function-call overhead for very small functions

→ Important point: `inline` is only a request
  → The compiler decides whether actual inlining is appropriate.

→ Modern meaning
  → The keyword also allows an identical function definition to appear in multiple translation units, which is important for functions defined in header files

```
inline double square(double value)
{
    return value * value;
}
```

Function Overloading

→ Several functions may have the same name if they have different parameter lists

→ The compiler selects the best matching overload

→ Overloads may differ in
- → Number of parameters
- → Parameter types
- → Parameter order

→ They cannot differ only in
- → Return type
- → Parameter names

```
int larger(int a, int b);
double larger(double a, double b);


larger(10, 20); // Calls int version
larger(1.5, 2.5); // Calls double version
```

Recursion

→ A recursive function calls itself

→ Every recursive solution requires:
  → A *base case* that stops recursion
  → A *recursive case* that moves toward the base case

→ Without a valid stopping condition, recursion continues until resources are exhausted

```
unsigned long long factorial(unsigned int n)
{
    if (n <= 1)
        return 1;

    return n * factorial(n - 1);
}
```

Recursion and the Call Stack

→ Every recursive call creates a new function invocation with:
  → Its own parameters
  → Its own local variables
  → Its own return location

→ This information is stored on the *call stack*

```
unsigned long long factorial(unsigned int n)
{
    if (n <= 1)
        return 1;

    return n * factorial(n - 1);
}
```

→ Suitable applications
  → Tree traversal
  → Divide-and-conquer algorithms
  → Naturally recursive data structures
  → Recursive mathematical definitions

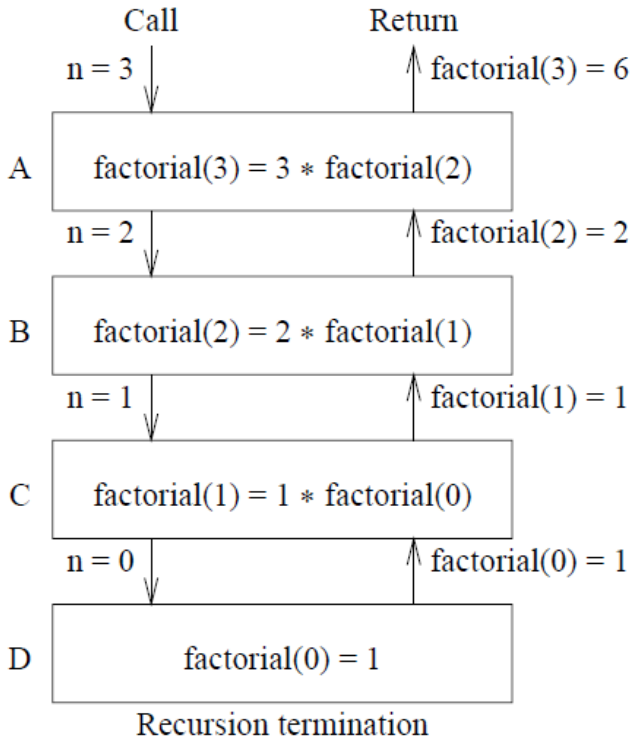
→ Limitations
  → Call overhead
  → Additional stack usage
  → Risk of stack overflow
  → A loop may be simpler and more efficient

Recursion and the Call Stack

```
unsigned long long factorial(unsigned int n)
{
    if (n <= 1)
        return 1;

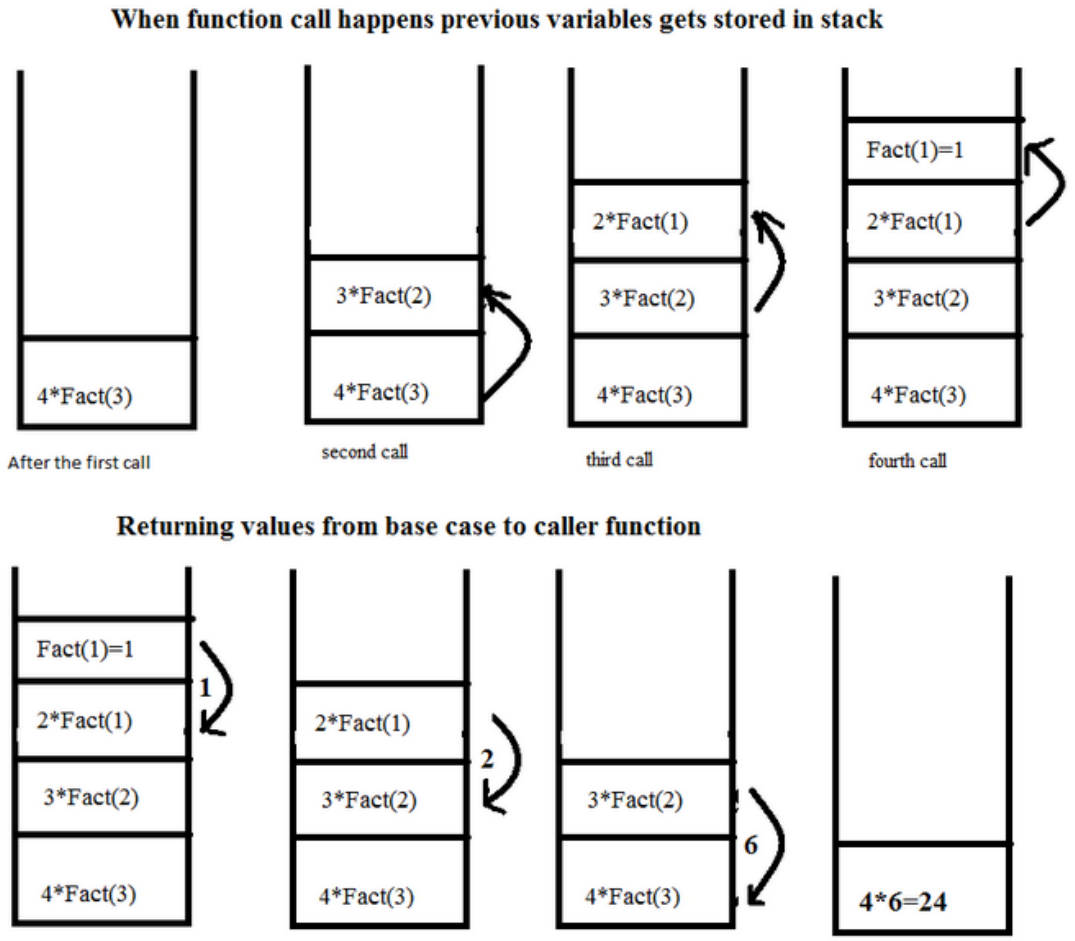
    return n * factorial(n - 1);
}
```

- → Every recursive call creates a new function invocation with:
  - → Its own parameters
  - → Its own local variables
  - → Its own return location
- → This information is stored on the call stack
- → Suitable applications
  - → Tree traversal
  - → Divide-and-conquer algorithms
  - → Naturally recursive data structures
  - → Recursive mathematical definitions



Recursion and the Call Stack

- → Every recursive call creates a new function invocation with:
  - → Its own parameters
  - → Its own local variables
  - → Its own return location
- → This information is stored on the call stack
- → Suitable applications
  - → Tree traversal
  - → Divide-and-conquer algorithms
  - → Naturally recursive data structures
  - → Recursive mathematical definitions

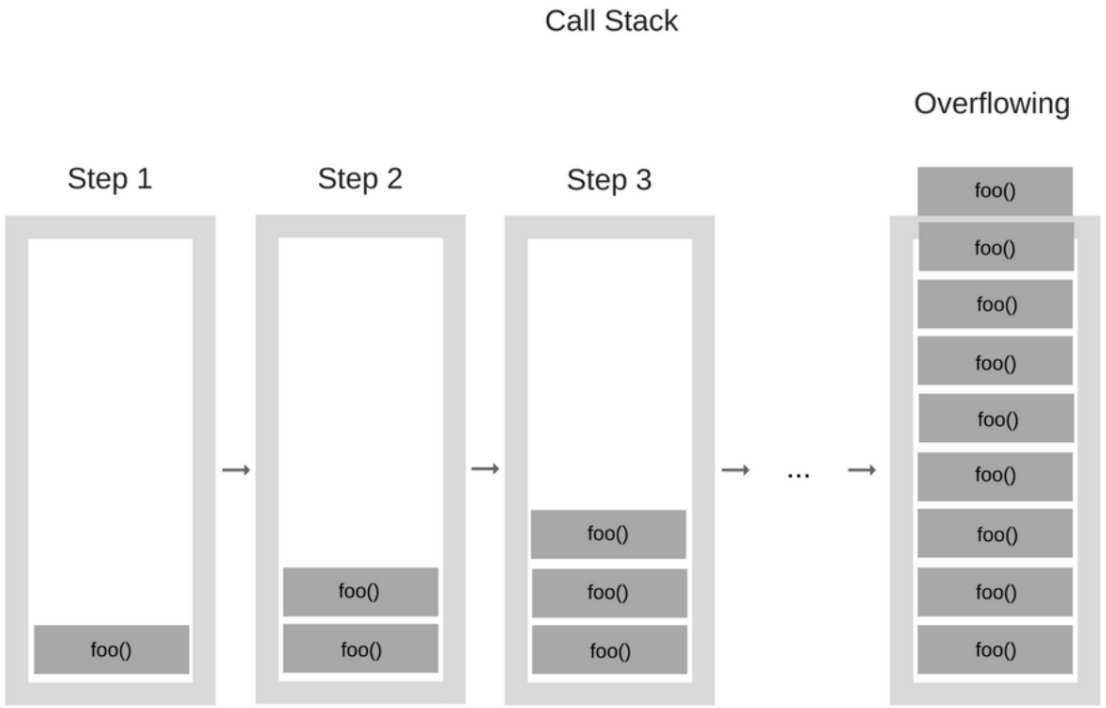


Recursion and the Call Stack

→ Every recursive call creates a new function invocation with:
  → Its own parameters
  → Its own local variables
  → Its own return location

→ This information is stored on the call stack

→ Suitable applications
  → Tree traversal
  → Divide-and-conquer algorithms
  → Naturally recursive data structures
  → Recursive mathematical definitions



Recursion and the Call Stack

→ Every recursive call creates a new function invocation with:
  → Its own parameters
  → Its own local variables
  → Its own return location

→ This information is stored on the *call stack*

```
unsigned long long factorial(unsigned int n)
{
    if (n <= 1)
        return 1;

    return n * factorial(n - 1);
}
```

→ Suitable applications
  → Tree traversal
  → Divide-and-conquer algorithms
  → Naturally recursive data structures
  → Recursive mathematical definitions

→ Limitations
  → Call overhead
  → Additional stack usage
  → Risk of stack overflow
  → A loop may be simpler and more efficient

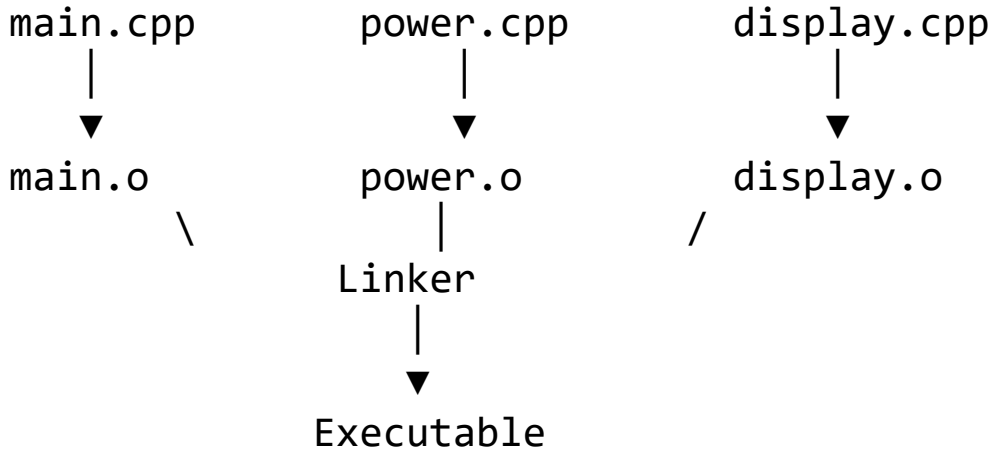
C++ Program Files

From Source Files to an Executable

→ A program can contain multiple `.cpp` source files

→ Build process

1. The preprocessor processes each source file
2. The compiler translates each resulting *translation unit*
3. An object file is produced for each source file
4. The linker combines object files and required libraries
5. The result is the executable program



→ Each `.cpp` file is compiled independently

Translation Units

→ A translation unit consists of:
- → One source file
- → The contents of all headers included by that source file
- → The result of preprocessing directives

→ Conceptually, preprocessing replaces the `#include` directives with the contents of the corresponding headers

→ The compiler processes the complete translation unit, not the original `.cpp` file in isolation

```
// main.cpp
#include <iostream>
#include "Power.h"

int main()
{
    std::cout << power(2.0, 8);
}
```

```
main.cpp + included headers → translation unit
```

Declarations and Definitions

→ A declaration tells the compiler that a name and type exist

```
double power(double x, int n);
```

→ A definition supplies the implementation or allocates storage

```
double power(double x, int n)
{
    // Implementation
}
```

Declarations and Definitions

→ Typical organization

```
// Power.h
double power(double x, int n);

// Power.cpp
#include "Power.h"

double power(double x, int n)
{
    // Implementation
}
```

→ This organization separates the public interface from the implementation

```
// main.cpp
#include "Power.h"
```

Linkage

→ Linkage determines whether the same name used in different scopes or translation units refers to the same entity

→ Three important possibilities
  → *No linkage*: the name is usable only in its local scope
  → *Internal linkage*: the name belongs to one translation unit
  → *External linkage*: the same entity can be accessed from several translation units

→ Function parameters and local variables have no linkage

→ Functions defined at namespace scope normally have external linkage

External Functions

→ A function can be defined in one source file and called from another

→ The declaration allows `main.cpp` to compile
→ The linker later connects the call with the definition in `Power.cpp`

→ Better organization
  → Place the function declaration in a header
  → This avoids manually repeating the declaration in every source file

```
// Power.cpp
double power(double x, int n)
{
    // ...
}

// main.cpp
double power(double x, int n);
// Declaration

int main()
{
    auto result{power(2.0, 10)};
}
```

```
// main.cpp
#include "Power.h"
```

External Variables

→ A variable can be declared without defining it by using `extern`

→ Exactly one source file provides the definition

→ Other source files include the declaration

```
// Configuration.h
extern int operating_mode;

// Configuration.cpp
int operating_mode{1};

// main.cpp
#include "Configuration.h"

void update()
{
    operating_mode = 2;
}
```

→ Important distinction

```
extern int operating_mode;  // Declaration
int operating_mode{1};      // Definition
```

Internal Names

→ Sometimes an entity should only be visible within one translation unit

→ An *unnamed namespace* provides internal linkage

→ These names can be used within the current translation unit but do not conflict with identically named entities in other source files

→ Use case
→ Keep implementation details private to one `.cpp` file

```
namespace
{
    constexpr int buffer_size{256};

    void initialize_hardware()
    {
        // Private implementation
    }
}
```

The One Definition Rule

→ The One Definition Rule (ODR) controls how declarations and definitions may appear

→ General principles
  → An entity must normally have only one definition in a translation unit
  → Functions and variables with external linkage should have one program-wide definition
  → A declaration may appear in several translation units
  → Certain inline entities may be defined in multiple translation units when their definitions are equivalent

→ Why this matters
  → Violating the rule may produce:
    → Compiler errors
    → Linker errors
    → Undefined behavior

→ The ODR explains why headers usually contain declarations rather than ordinary function definitions

Preprocessing
C PreProcessor (CPP)

Preprocessing

→ Preprocessing happens before compilation
→ A preprocessing directive begins with #

→ Common uses
  → Include header contents
  → Define macros
  → Select code conditionally
  → Test compiler or platform characteristics
  → Support debugging configurations

→ The preprocessor performs textual processing and does not fully understand C++ types or scope

```
#include <iostream>
#define BUFFER_SIZE 256

#if defined(DEBUG)
...
#endif
```

Object-Like Macros

→ An object-like macro defines a textual replacement:

```
#define BUFFER_SIZE 256
```

→ Before compilation:

```
int buffer[BUFFER_SIZE];
```

→ It is transformed approximately into:

```
int buffer[256];
```

→ Problems with macros
- → No type checking
- → No C++ scope
- → Cannot be placed naturally in a namespace
- → May produce confusing diagnostics

→ Prefer a C++ constant:

```
constexpr std::size_t BUFFER_SIZE{256};
```

Function-Like Macros

→ A macro can look similar to a function:

```
#define MAX(A, B) ((A) >= (B) ? (A) : (B))
```

→ But it is only textual substitution:
  → May evaluate `calculate()` more than once because `A` appears several times in the replacement text

```
MAX(calculate(), 0)
((calculate()) >= (0) ? (calculate()) : (0))
```

→ Additional risks
  → Repeated evaluation
  → Operator-precedence errors
  → No parameter types
  → No return type
  → Difficult debugging
  → Unexpected behavior with side effects

Undefining Macros

→ A macro remains defined from its `#define` directive onward

→ It can be removed with `#undef`

→ After `#undef`, the identifier is no longer treated as that macro

```
#define DEBUG_OUTPUT

// Code using DEBUG_OUTPUT

#undef DEBUG_OUTPUT
```

```
#define PI 3.141592653589793
// PI is replaced here

#undef PI
// PI is no longer replaced here
```

Including Header Files

→ Standard Library headers use angle brackets

```
#include <iostream>
#include <vector>
```

→ Your own headers normally use quotation marks

```
#include "Power.h"
#include "Configuration.h"
```

→ Headers primarily contain:
- → Function declarations
- → Type definitions
- → Template definitions
- → Constants
- → Inline definitions

→ Unnecessary includes increase compilation work and can confuse dependencies

Multiple Inclusion Problem

→ One header may be included several times indirectly

→ Without protection, `Configuration.h` may appear twice in the same translation unit

→ This can cause:
- → Duplicate definitions
- → ODR violations
- → Compilation errors
- → Recursive inclusion

```
main.cpp
├── Controller.h
│   └── Configuration.h
└── Device.h
    └── Configuration.h
```

Include Guards

→ An include guard ensures that a header's contents are processed only once per translation unit

→ How it works

1. `#ifndef` tests whether `POWER_H` is undefined
2. On the first inclusion, the condition is true
3. `#define POWER_H` marks the header as processed
4. Later inclusions skip the guarded content

```
#ifndef POWER_H
#define POWER_H

double power(double x, int n);

#endif
```

→ Each header should use a unique guard identifier

#pragma once

→ Many compilers support it

→ It serves the same practical purpose as an include guard by preventing repeated inclusion

→ Trade-off
- → Shorter and less error-prone than a manual guard
- → Supported by nearly all common compilers
- → Not part of standard C++

```
#pragma once

double power(double x, int n);
```

```
#ifndef POWER_H
#define POWER_H

double power(double x, int n);

#endif
```

Example Header File

CLIENT

```
// main.cpp
#include <iostream>
#include "Power.h"

int main()
{
    std::cout << power(8.0, 3) << '\n';
}
```

HEADER

```
// Power.h
#ifndef POWER_H
#define POWER_H

double power(double x, int n);

#endif
```

IMPLEMENTATION

```
// Power.cpp
#include "Power.h"

double power(double x, int n)
{
    // Function implementation
}
```

Conditional Compilation

→ Conditional preprocessing selects which source text reaches the compiler

→ Common directives include

→ This technique can produce different builds from the same source code

```
#if CONDITION
    // Included when CONDITION is nonzero
#endif
```

```
#if
#ifdef
#ifndef
#elif
#else
#endif
```

```
#ifdef DEBUG
    std::cout << "Debug information\n";
#endif
```

Testing Macro Values

→ A macro can represent a configuration value

→ Code can be selected according to that value

→ Typical applications
- → Platform-specific code
- → Compiler-specific code
- → Debug and release configurations
- → Optional functionality
- → Hardware variants

→ Risk
- → Excessive conditional compilation creates many possible source variants and makes testing more difficult

```
#define LOG_LEVEL 2

#if LOG_LEVEL >= 2
    log_detailed_information();
#elif LOG_LEVEL == 1
    log_basic_information();
#else
    // Logging disabled
#endif
```

Debugging with Preprocessing

→ Diagnostic code can be included only in debugging builds

→ It can be removed from another build by leaving `DEBUG` undefined

→ Benefits
- → No runtime test in builds where code is excluded
- → Diagnostic output can be centralized
- → Release builds can omit debugging statements

→ Limitation
- → The debug and release builds contain different compiled code, so both configurations require testing

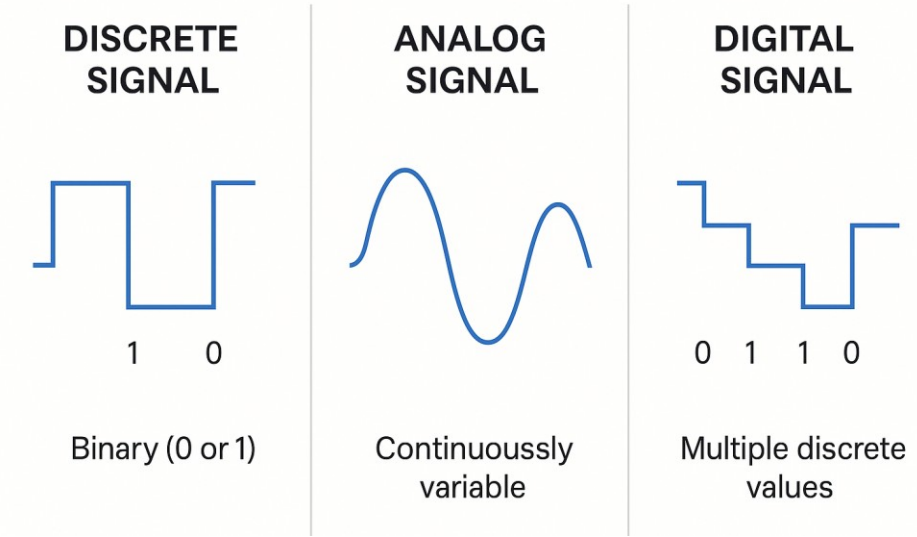
```
#ifdef DEBUG
    std::cerr << "value = " << value << '\n';
#endif
```

Analog Signals vs Digital Signals

Signals in Embedded Systems

→ Embedded systems interact with the physical world through signals
  → Examples: temperature sensor, light sensor, potentiometer, push button

→ Signals can be classified into two categories:
  → **Analog Signals**
    → Continuous values
    → Continuous time
    → Can take any value within a range
  → **Digital Signals**
    → Discrete values
    → Usually represented as 0 and 1
    → Processed directly by microcontrollers



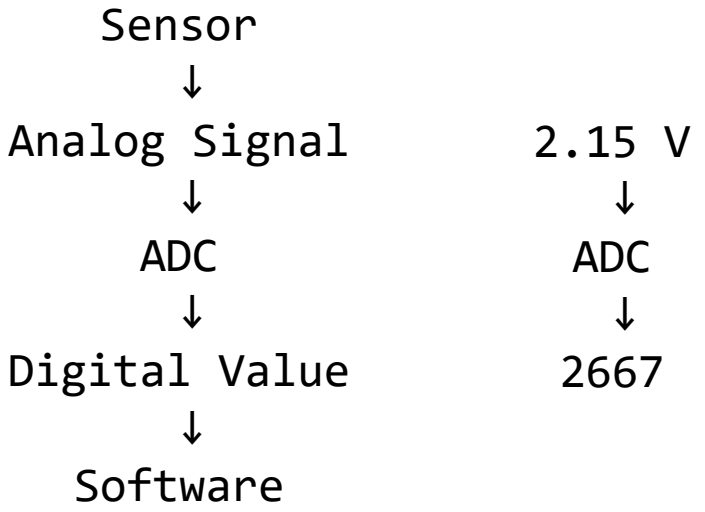
→ Many cyber-physical systems combine both analog and digital components

Why Convert Analog to Digital?

→ Microcontrollers are fundamentally digital devices

→ They cannot directly process:
- → Voltage levels
- → Sound waves
- → Temperature signals
- → Light intensity

→ Instead, an Analog-to-Digital Converter (ADC) converts an analog signal into a digital value

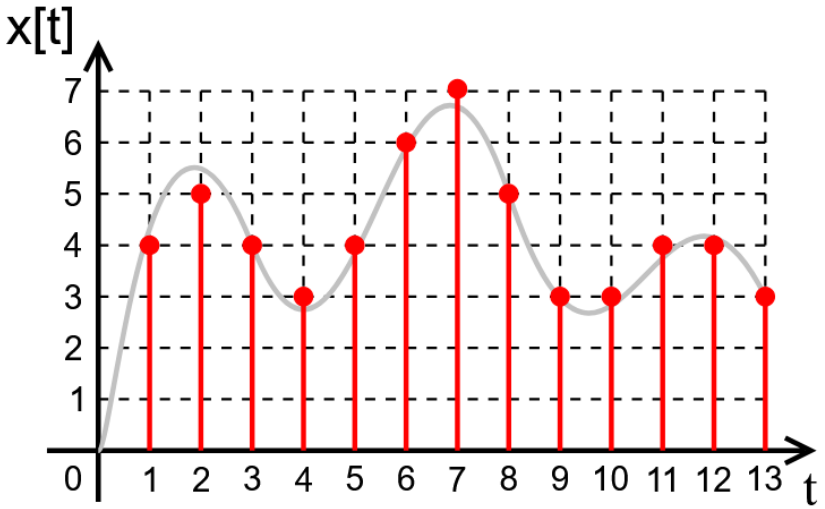


Sampling and Quantization

→ Converting an analog signal into digital form involves two steps

1. *Sampling*: measure the signal at discrete points in time (1000 samples / s)
2. *Quantization*: round each measurement to one of the available digital levels (8-bit – 256 levels, 16-bit - 65536)

→ Digital systems observe only a sequence of sampled measurements

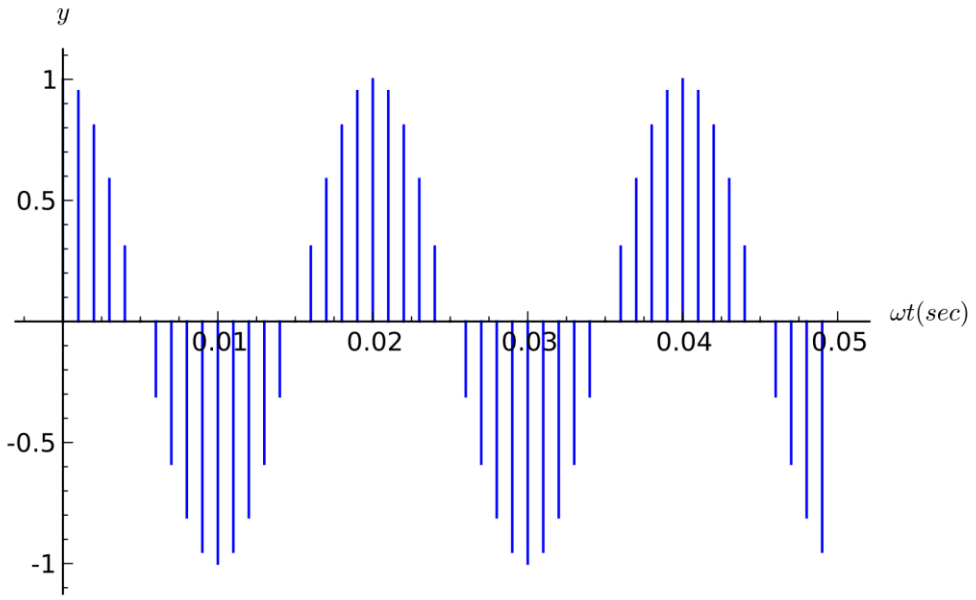


Sampling and Quantization

→ Converting an analog signal into digital form involves two steps

1. *Sampling*: measure the signal at discrete points in time (1000 samples / s)
2. *Quantization*: round each measurement to one of the available digital levels (8-bit – 256 levels, 16-bit - 65536)

→ Digital systems observe only a sequence of sampled measurements



Discrete cosine waveform with a frequency of 50 Hz and a sampling rate of 1000 samples/sec

Analog vs Digital Signals

→ The physical world is mostly analog, while microcontrollers are digital
→ Embedded systems bridge this gap using components such as ADCs, DACs, digital communication interfaces, and GPIOs

Analog	Digital
Continuous values	Discrete values
Infinite resolution	Limited resolution
More sensitive to noise	More noise tolerant
Represents physical phenomena directly	Easy to process with software
Requires ADC for microcontrollers	Directly usable by GPIOs

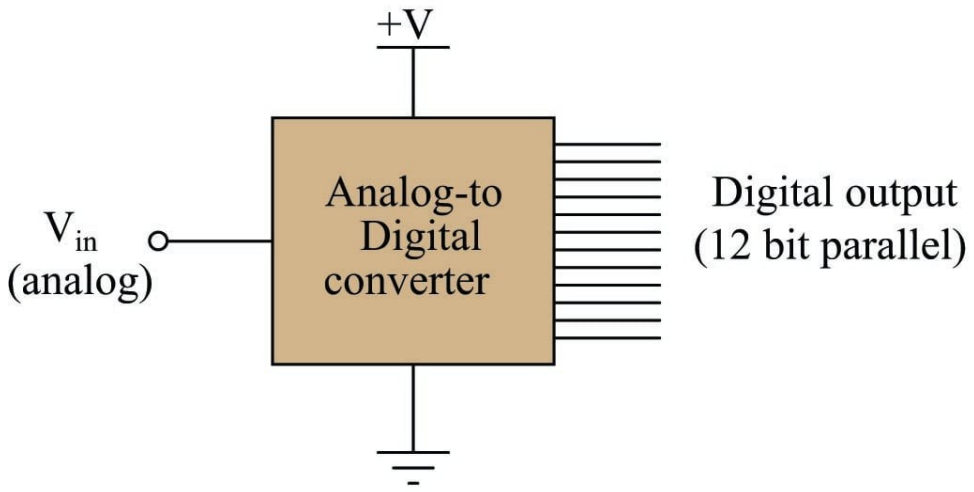
Analog-to-Digital Converter

What is an Analog-to-Digital Converter

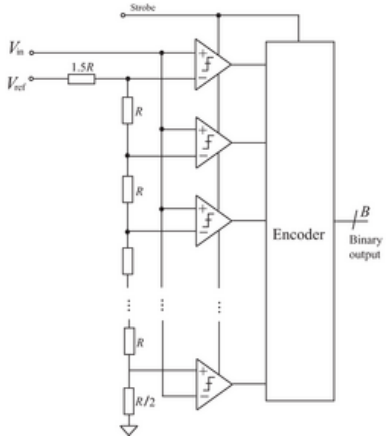
→ An ADC converts a continuously varying analog signal into a digital value that can be processed by a microcontroller

→ The Rpi Pico contains a built-in 12-bit ADC, allowing software to measure analog voltages and convert them into values between 0 and 4095

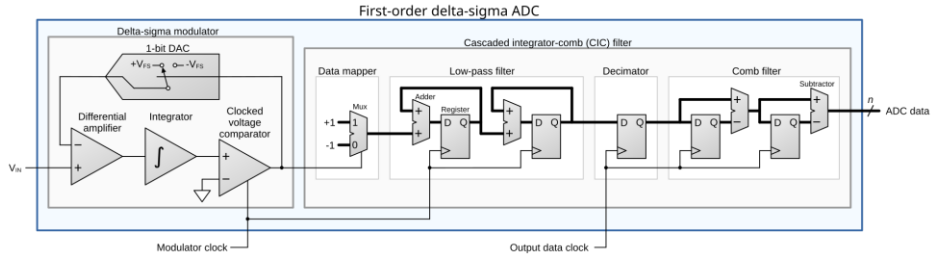
→ ADC resolution
- → ADC resolution determines how many distinct digital values can be represented: $N = 2^M$ where M is the ADC resolution in bits
- → Higher resolution allows smaller voltage changes to be distinguished

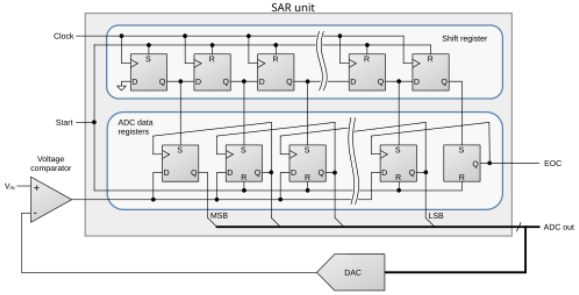


Types of ADCs



- → Flash ADC
  - → Extremely fast
  - → Requires many comparators
  - → Expensive and power-hungry
- → Sigma-Delta ADC
  - → Very high accuracy
  - → Lower sampling rates
  - → Common in precision measurements
- → Successive Approximation Register (SAR) ADC
  - → Good balance of speed and accuracy
  - → Widely used in microcontrollers
  - → Used by the RP2040 ADC





→ The SAR ADC is the most common ADC encountered in embedded systems.

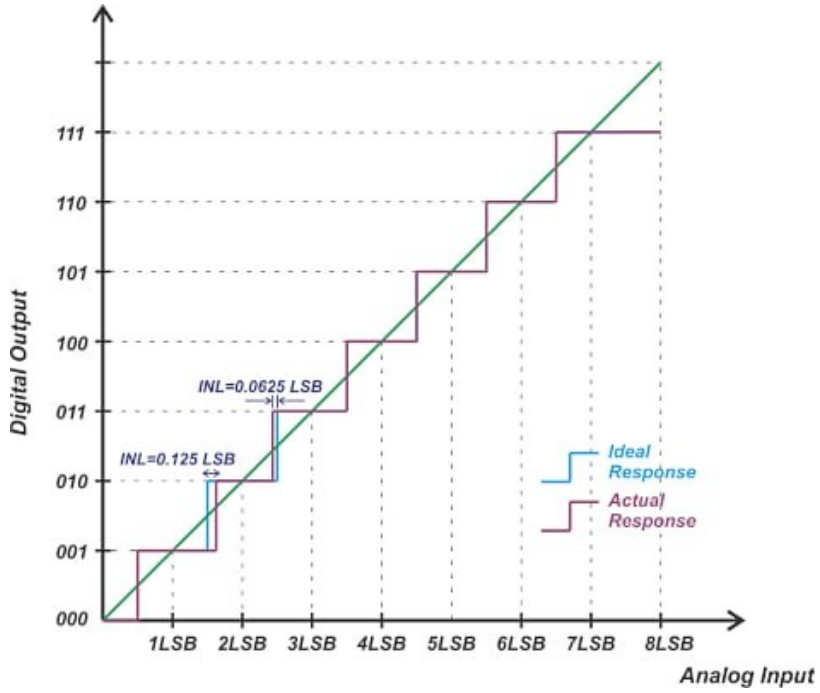
ADC Accuracy vs Resolution

→ Resolution and accuracy are not the same

→ Resolution: determined by the number of bits

→ Accuracy is affected by:
- → Noise
- → Offset errors
- → Gain errors
- → Non-linearity
- → Reference voltage quality

→ A 12-bit ADC may not actually deliver 12 useful bits of information



Effective number of bits

→ Effective Number of Bits (ENOB) measures the actual useful resolution of an ADC

→ It accounts for:
- → Noise
- → Distortion
- → Hardware imperfections

→ ENOB tells us how many bits are actually useful, not how many bits appear in the ADC register

ADC specification: 12 bits
ENOB = 10.5 bits

12-bit ADC
≈ 2^10.5
≈ 1450 useful levels

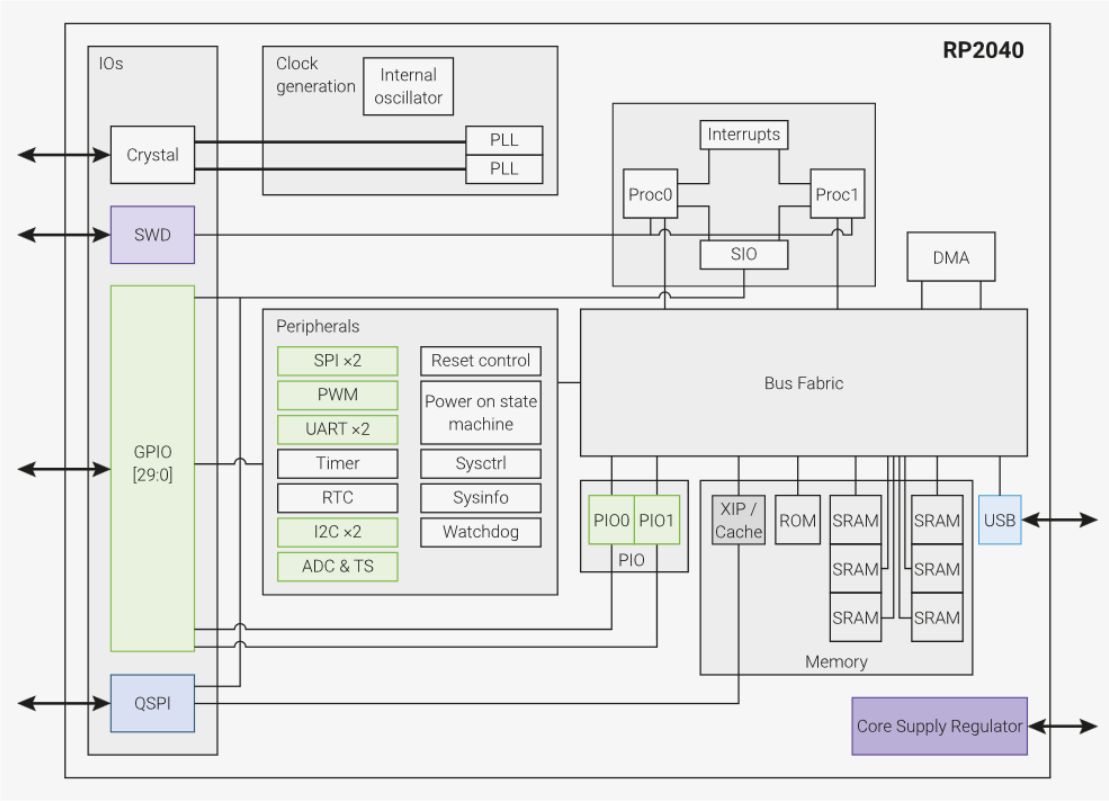
Compared to the ideal 4096 levels

Raspberry Pi Pico ADC

ADC on the RP2040

→ The RP2040 contains a built-in 12-bit Successive Approximation Register (SAR) ADC

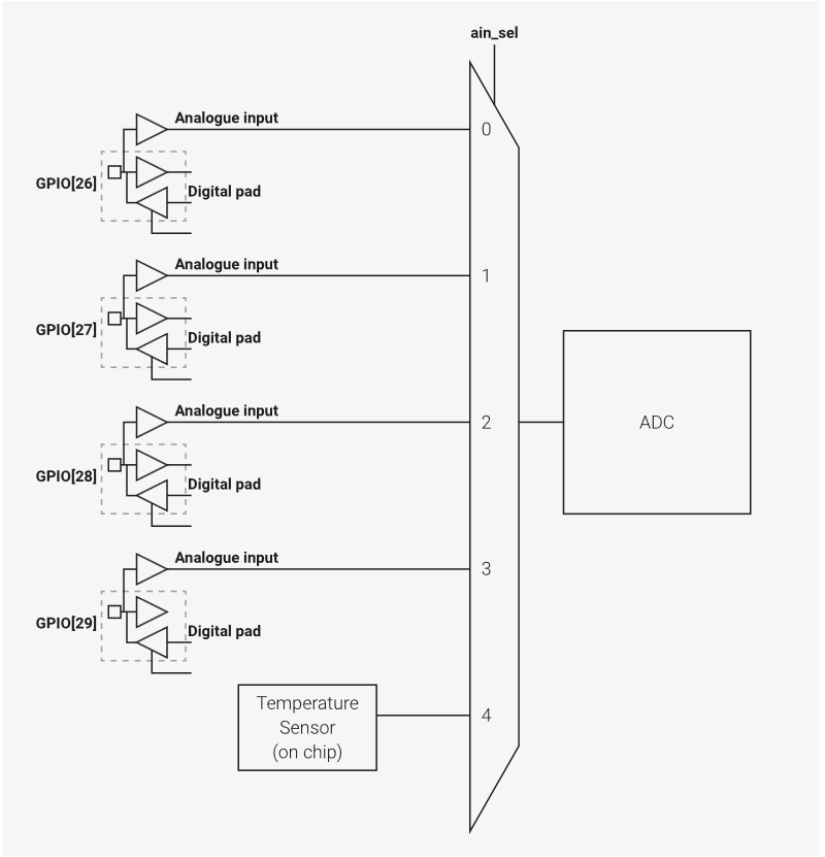
→ Features:
- → 500 kS/s
- → 12 bit (RP2040 8.7 ENOB)
- → 5 input mux
  - → 4 inputs on package pins shared with GPIO[29:26]
  - → 1 input for the internal temperature sensor
- → 8 element receive sample FIFO
- → Interrupt generation
- → DMA interface



ADC on the RP2040

→ The RP2040 contains a built-in 12-bit Successive Approximation Register (SAR) ADC

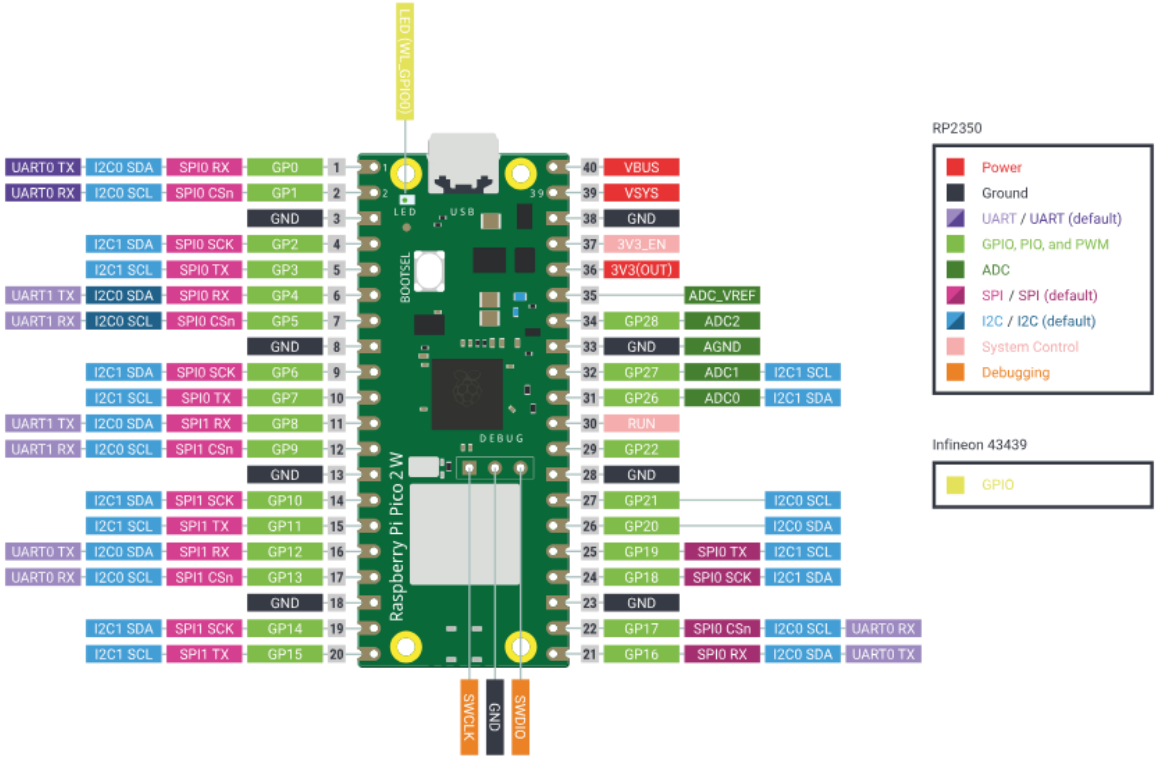
→ Features:
- → 500 kS/s
- → 12 bit (RP2040 8.7 ENOB)
- → 5 input mux
  - → 4 inputs on package pins shared with GPIO[29:26]
  - → 1 input for the internal temperature sensor
- → 8 element receive sample FIFO
- → Interrupt generation
- → DMA interface



ADC on the RP2040

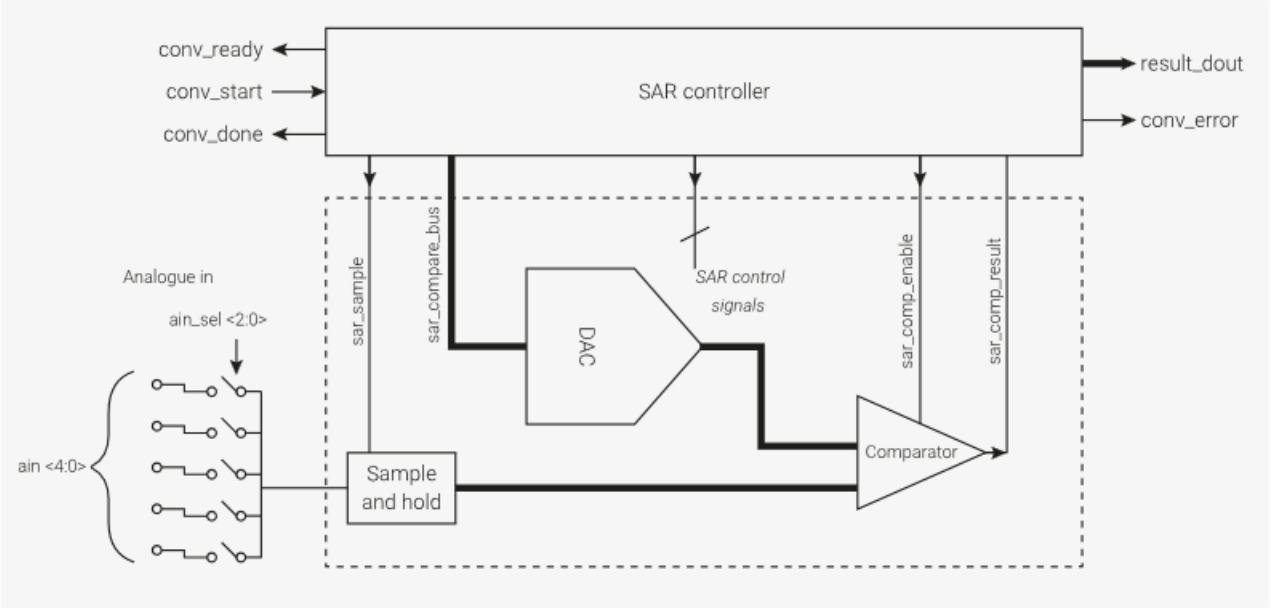
→ The RP2040 contains a built-in 12-bit Successive Approximation Register (SAR) ADC

→ Features:
- → 500 kS/s
- → 12 bit (RP2040 8.7 ENOB)
- → 5 input mux
  - → 4 inputs on package pins shared with GPIO[29:26]
  - → 1 input for the internal temperature sensor
- → 8 element receive sample FIFO
- → Interrupt generation
- → DMA interface



ADC Signal Path

```
Analog Signal
      ↓
GPIO26-29
      ↓
Input Multiplexer
      ↓
12-bit SAR ADC
      ↓
ADC Result Register
      ↓
Software
```



→ Only one channel is converted at a time

→ The software selects which input channel should be connected to the ADC

Using the Pico SDK ADC Library

Typical sequence:

→ Include library
→ Initialize the ADC hardware
→ Configure a GPIO pin for analog input
→ Select the ADC channel
→ Perform a conversion
→ Return the result

```
#include "pico/stdlib.h"
#include "hardware/adc.h"

int main()
{
    stdio_init_all();

    adc_init();
    adc_gpio_init(26);
    adc_select_input(0);

    while(true)
    {
        uint16_t value = adc_read();

        printf("ADC = %u\n", value);

        sleep_ms(100);
    }
}
```

Converting ADC Counts to Voltage

→ The ADC returns a value between: `0 ... 4095`

→ Voltage can be estimated as:
```
voltage = adc_value * 3.3 / 4095.0;
```

→ Example:
```
ADC = 2048
Voltage ≈ 1.65 V
```

→ For a 12-bit ADC:
```
1 LSB ≈ 0.805 mV (under ideal conditions)
1 LSB ≈ 6.445 mV (in reality, 9 ENOB)
```

→ A 12-bit ADC does not necessarily provide 12 useful bits because noise and hardware imperfections reduce the Effective Number of Bits

Announcements

Student Collaboration Day Sønderborg

October 6th 2026 kl 14-17

Plenty of time – so teaching AND networking can be combined

Activities:

Fair: 31 companies ready to discuss future collaboration on projects, internships and jobs

Ad boards: Physical ads (jobs/projects) from the participating companies

Career counselling and free professional photo for CV

Wanna join?

Just show up from 14.00 and pick up a study sticker at the reception area, register your attendance online – and start networking….

Participant list here:



SDU

Git Workshop
6 Oct, 16:30, U109



- → Git fundamentals
- → Repositories, commits, and branches
- → Working with GitHub
- → Resolving simple conflicts
- → Best practices for student projects

Questions 

Lecturer, Teaching Assistant and Instructors



Krzysztof Sierszecki
krzys@mmmi.sdu.dk

Lecturer



Svens Gotvoni
svgot23@student.sdu.dk

Teaching Assistant

Instructors

1. Mihnea-Stefan Tudor
2. Rafal Kamil Fuchs
3. Vlad Slimovschi

Exercise 5 